SmartTAR

Version 1.4.0

Technical Architecture, Capabilities and Operating Model

STAR archive format | Additive generations | Global deduplication
Canonical manifest | Verification | Browse | Extraction | Transaction safety

Document status English-only technical documentation updated to include the observed backward-extraction behavior of additive archives.

Validated design: 25 July 2026
Compatibility update: 26 July 2026

Contents

Scope This document explains the released format and operating behavior. It does not describe the complete source-code API. Block counts and storage savings depend on the selected data and compression method.

1. Executive overview

SmartTAR is a Windows-oriented archival and backup application that writes a portable .star file built from standard TAR-compatible outer records and independently compressed data blocks. Version 1.4.0 extends the original STAR model with additive generations, global file-level deduplication, multi-root browsing and a canonical single-manifest update model.

Design principle Existing data blocks are immutable. Each ADD operation appends only new unique data and metadata. Byte-identical files are represented as manifest aliases that point directly to an existing physical file target.

Capability	Summary
Portable single file	Archive data, block metadata and the current manifest are contained in one .star file.
Mixed compression generations	Each new ADD operation may independently use Balanced, Smart, Solid or Store.
Global deduplication	Every non-empty byte-identical file can be represented as an alias, regardless of path, generation or selected method.
Selective recovery	Browse or extract a file, folder, ADD batch, ADD root or the complete archive.
Verification	Checks block presence, TAR readability, SHA-256, path safety, alias targets and target sizes.
Transactional ADD	Builds and verifies a temporary result before replacing the original archive.
Backward extraction	Observed SmartTAR 1.3.2 behavior confirms that a 1.4.0 additive archive remains extractable because the archive uses one readable manifest and an ordinary isolated ADD directory tree.

2. Core capabilities and compression modes

- Create STAR archives from a file or directory using Balanced, Smart, Solid or Store mode.
- Append independent timestamped ADD generations without recompressing existing blocks.
- Deduplicate identical non-empty files globally by size and SHA-256 content identity.
- Keep logical copies and historical ADD batches while storing each unique payload only once.
- Browse multi-root archives and restore selected items without extracting everything.
- Perform full extraction to a primary folder and a separate <primary>_add folder in version 1.4.0.
- Generate persistent TXT reports for Create, ADD, Extract and Verify operations.
- Read earlier released STAR archives and preserve extraction readability for additive archives where older readers accept the unified manifest and standard directory paths.

Mode	Behavior	Typical use
Balanced	Mixed blocks selected from file characteristics; balances speed and ratio.	General archives and mixed application folders.
Smart	Deeper content analysis and maximum-compression preference.	When ratio matters more than creation time.
Solid	A single solid data stream where applicable.	Sets with many similar small files.
Store	No payload compression.	Already compressed data, fast capture or controlled testing.

The selected compression mode applies only to newly stored unique data. If an ADD generation contains only duplicates, no new payload block is required. The generation still receives structural metadata and aliases.

3. STAR archive architecture

```
Outer .star container
  blocks/000001_structure.tar
  blocks/000002_compressible.tar.xz
  blocks/000003_structure_add001.tar
  ...
  manifest.json    <- exactly one current manifest, final logical entry
```

The outer container separates structure from payload groups. Data blocks are independently addressable and carry IDs, paths, source-byte counts, file counts, compression methods and SHA-256 values in the manifest. This supports selective streaming, verification and append-only growth.

Layer	Responsibility
Outer STAR/TAR	Packages blocks and the final manifest into one portable file.
Structure blocks	Represent directory trees and empty or structural entries.
Data blocks	Store physically unique file payloads using the selected method.
Manifest	Describes active blocks, roots, aliases, histories, summaries and the canonical tail layout.
Alias catalog	Maps a logical file path to one physically stored target path and records its byte length.

4. Archive creation workflow

```
Analyze source -> classify files -> detect duplicates -> build structure
-> create data blocks -> write canonical manifest -> publish
```

During initial creation, SmartTAR analyzes the source, groups files, omits duplicate payloads from data blocks and creates a manifest alias for each duplicate. The canonical manifest is written as the final logical outer entry. The manifest includes an outerLayout marker that enables safe replacement during future ADD operations.

5. ADD generation workflow

```
Read current manifest
Build deduplication index from physical block members
Prepare ADD/ADD_yyyyMMdd_HHmmss/<source>/...
Deduplicate against all physical files
Compress only remaining unique data
Copy original STAR to a transaction file
Remove the old manifest tail from the temporary copy
Append new blocks and one merged manifest
Verify temporary archive
Replace original only after Verification: OK
```

Important Existing aliases are not materialized when building the ADD deduplication index. This

prevents alias-to-alias chains. Every new alias must target a physically stored file.

6. Global file deduplication

SmartTAR 1.4.0 uses full-file content identity. Non-empty files are first grouped by exact byte length. SHA-256 is calculated only for same-size candidates. A matching file is omitted from the new data block and represented by a manifest alias.

```
Logical path:   ADD/ADD_20260725_235138/Test/example.bin
Physical target: original-root/example.bin
Manifest record: path + target + bytes
```

Property	Effect
Path independent	Renaming or moving a duplicate does not prevent deduplication.
Generation independent	A duplicate in any later ADD generation can reuse the original payload.
Method independent	Balanced, Smart, Solid and Store generations can reuse the same target.
Non-empty files	The deduplication threshold is 1 byte. Zero-byte files remain ordinary entries.
Direct targets only	Alias-to-alias chains are rejected during manifest assembly and Verify.

Validated example A 51.30 MB folder containing 69 non-empty files was added again. All 69 files became aliases, the archive grew by approximately 12.50 KB, and Verify reported all aliases and blocks as valid. The remaining growth was structural and manifest overhead.

7. Canonical single manifest

New STAR archives contain one current manifest.json as the final logical outer entry. The manifest stores an outerLayout section with a validated truncateOffset that marks the beginning of the replaceable tail record.

```
"outerLayout": {
 "schema": 1,
 "mode": "replaceable-tail-manifest",
 "manifestEntry": "manifest.json",
 "manifestPosition": "last",
 "truncateOffset": <byte offset>,
 "alignmentBytes": 512
}
```

During ADD, SmartTAR copies the original archive to a uniquely named transaction file, validates the layout and offset, truncates only the temporary copy, appends new blocks, calculates a new offset, writes one merged manifest, verifies the result and only then replaces the original archive.

Compatibility effect Keeping metadata in one readable manifest and representing ADD history as normal isolated directory paths allows older extraction logic to continue reconstructing the archive tree, even when the older application does not understand the newer presentation rules.

8. Browse, extraction and restoration layout

```
Logical archive view
  PrimaryRoot/
  ADD/
    ADD_yyyyMMdd_HHmmss/
      SourceName/
        files and folders

Full extraction in SmartTAR 1.4.0
  Destination/PrimaryRoot/
  Destination/PrimaryRoot_add/
    ADD_yyyyMMdd_HHmmss/SourceName/...
```

Selection	Result in SmartTAR 1.4.0
Primary root	Restores only the original primary tree.
ADD	Restores the ADD root as <primary>_add.
Timestamped batch	Restores only the selected ADD_yyyyMMdd_HHmmss directory.
Source folder or subfolder	Restores only the selected leaf and descendants.
Single file	Streams only the required block or physical alias target.

Older extraction behavior SmartTAR 1.3.2 extracts the same logical content but writes the additive root as a generic ADD folder. The timestamped ADD_yyyyMMdd_HHmmss subdirectories remain intact, so generations stay separated and no payload is lost.

9. Verification and transaction safety

- Every manifest block must exist at a safe relative path.
- Each block must be readable as a TAR payload and match its recorded SHA-256.
- Unsafe archive entries and path traversal are rejected.
- Every alias path and target must be safe.
- Every alias target must be physically present in blocks, not another alias.
- The physical target length must match alias.bytes.
- Canonical archives must contain exactly one current manifest as the final logical entry.
- ADD publication occurs only after the temporary archive returns Verification: OK.

Control	Purpose
Destination-local temporary copy	Protects the original STAR from partial ADD writes.
Unique temporary name	Uses archive.star.adding.<GUID>.tmp to avoid collisions.

Control	Purpose
Free-space precheck	Estimates archive + source + safety reserve before the transaction.
Retry cleanup	Removes failed or cancelled transaction files after worker termination.
Short work paths	Reduces Windows PowerShell 5.1 MAX_PATH pressure.
Stream disposal	File streams use try/finally or C# using blocks.

10. Backward extraction compatibility

Testing performed on 26 July 2026 showed that SmartTAR 1.3.2 successfully extracted a SmartTAR 1.4.0 archive containing ADD generations. This updates the earlier assumption that an additive archive necessarily required version 1.4.0 for extraction.

Observed result SmartTAR 1.3.2 extracted both the primary content and all ADD generations correctly. The only visible difference was the destination name of the additive root: version 1.4.0 used <primary>_add, while version 1.3.2 used ADD.

```
SmartTAR 1.4.0 output
  PrimaryRoot/
  PrimaryRoot_add/
    ADD_20260726_001113/
    ADD_20260726_001147/
    ADD_20260726_001229/

SmartTAR 1.3.2 output
  PrimaryRoot/
  ADD/
    ADD_20260726_001113/
    ADD_20260726_001147/
    ADD_20260726_001229/
```

The result is explained by two format decisions: the archive uses one unified JSON manifest, and additive content is isolated under the ordinary ADD directory with one timestamped subdirectory per generation. An older extractor can therefore follow the manifest paths without understanding the newer naming or presentation logic.

Reader / function	1.4.0 archive before ADD	1.4.0 archive after ADD
SmartTAR 1.4.0	Supported	Supported, including native multi-root naming and all 1.4.0 functions.
SmartTAR 1.3.2 full extraction	Supported	Observed as successful. ADD content is written under a generic ADD root.
Earlier SmartTAR versions	Expected where the reader accepts the unified manifest and standard paths.	Theoretically extractable for the same reason, but not yet validated for every release.
ADD, Browse and Verify using older versions	Version-dependent.	Not implied by extraction compatibility; newer metadata and workflows may still require 1.4.0.

Compatibility rule A SmartTAR 1.4.0 additive archive is backward-readable at the extraction level when the older reader can parse the unified manifest and extract ordinary paths. Successful extraction does not guarantee that older versions can create ADD generations, present multi-root browsing, perform 1.4.0 verification rules or preserve the 1.4.0 destination naming convention.

A timestamp-generated ADD directory name is normally unique. A theoretical collision remains possible if two ADD operations receive the same timestamp at the available precision. Sequential processing makes this unlikely in normal use; a future implementation may add sub-second precision or a numeric suffix if stricter uniqueness is required.

11. Operational guidance

- Run Verify after important ADD operations and before relying on an archive as the only copy.
- Keep sufficient free space for the transaction copy, new data and a safety reserve.
- Do not manually edit manifest.json or blocks inside a STAR archive.
- Keep TXT reports when diagnosing unexpected behavior.
- Use Browse for selective restore and full Extract for complete recovery.
- When using an older SmartTAR version for recovery, expect the additive root to be named ADD rather than <primary>_add.
- Do not treat backward extraction success as proof that all 1.4.0 operations are supported by the older application.
- If an operation is cancelled, confirm that the original archive still passes Verify.

12. Summary

SmartTAR 1.4.0 combines a portable single-file archive, independently compressed blocks, global file deduplication, independent ADD generations, one replaceable canonical manifest, selective restoration and integrity verification. The unified manifest and isolated ADD directory structure also provide practical backward extraction compatibility: SmartTAR 1.3.2 has been observed to extract a 1.4.0 additive archive correctly, with only a different name for the outer ADD destination folder. Earlier versions may theoretically behave the same way, but should be described as unvalidated until tested.

Document status This document describes the validated SmartTAR 1.4.0 design as of 25 July 2026 and the backward-extraction finding observed on 26 July 2026. Implementation details may change in future versions while format compatibility is preserved.

SmartTAR 1.4.0

13. Adaptive parallel block compression

This section updates the SmartTAR 1.4.0 technical documentation with the optimized block-build pipeline prepared on 26 July 2026. The change affects archive creation performance only. STAR block formats, the canonical manifest model, extraction behavior, ADD compatibility and backward readability remain unchanged.

Implementation status The scheduler has been integrated into the working SmartTAR 1.4.0 PowerShell build. Runtime benchmarking on Windows is still recommended before performance gains are treated as validated.

13.1 Build pipeline

After analysis, classification and deduplication, eligible data groups receive deterministic block IDs. Independent blocks may then be compressed concurrently. The structure block remains sequential, and completed data blocks are published to the outer STAR container sequentially in ID order.

```
Analyze and classify -> assign deterministic block IDs -> build independent blocks in parallel
-> validate completed block files -> publish blocks sequentially by ID -> write one canonical
manifest
```

Control	Implemented behavior
CPU scheduling	Worker limit is approximately 50% of logical processors, with a minimum of 1 and an initial maximum of 4 workers.
Memory protection	Before starting another worker, SmartTAR preserves the larger of 2 GB or 20% of total physical RAM as a system reserve.
Worker estimates	Planning reserves: XZ 1.5 GB, Zstandard 1.0 GB, Store 256 MB per active worker.
Process priority	Compression processes use BelowNormal priority so interactive Windows tasks retain preference.
Outer STAR writes	Workers never append concurrently to the same outer archive. Publishing remains sequential and deterministic.
Failure handling	A failed parallel group retains the proven sequential chunk fallback path.
Solid mode	A true Solid profile remains a single data block and normally has no block-level parallelism.

13.2 RAM, SSD and staging behavior

The new scheduler preserves the existing SmartTAR resource model. Compression still uses RAM in the same manner as the previous 1.4.0 build. Parallelism only allows more than one independent bsdtar process to operate when CPU and memory limits permit.

• **RAM usage:** Existing streaming and in-memory behavior remains unchanged. The scheduler adds only a safety gate before another compressor is started.

• **Swap avoidance:** If the reserve would be crossed, SmartTAR does not suspend or terminate an active compressor. It delays the next job until a worker finishes and memory is checked again.

• **SSD writes:** Staging continues to prefer hard links. Copy fallback remains available only where hard links cannot be created.

• **Working set:** Only stages belonging to active workers are needed. SmartTAR does not prepare every group as a full copied stage in advance.

• **Temporary blocks:** More than one completed or active block may briefly exist in the hidden .stw work directory, but each block remains independent and is removed after publication.

• **ADD:** ADD generation calls the same compression pipeline, so safe parallel block creation is inherited automatically.

Resource rule Maximize useful parallel work without intentionally driving Windows into paging. CPU availability permits concurrency; physical-memory availability can reduce it at any time before a new worker starts.

13.3 Compatibility and validation

Parallel creation does not require a new STAR format version because completed blocks are byte-addressable archive members with the same metadata as sequentially created blocks. The manifest records the active pipeline as `parallel-block-build-sequential-publish`. Older readers do not need to understand how a block was produced.

Recommended Windows validation: create representative Balanced and Smart archives, confirm that multiple bsdtar processes appear when two or more eligible groups exist, run Verify, extract with SmartTAR 1.4.0 and 1.3.2, compare restored hashes, and observe CPU, available RAM and page-file activity.